

(전자 2 – B08)



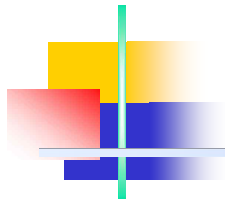
Feature Channel Restoration and Average Pooling-Based Channel Resize for FCM

2026. 6. 16(화)

방준혁, 김민



한국항공대학교
Korea Aerospace University



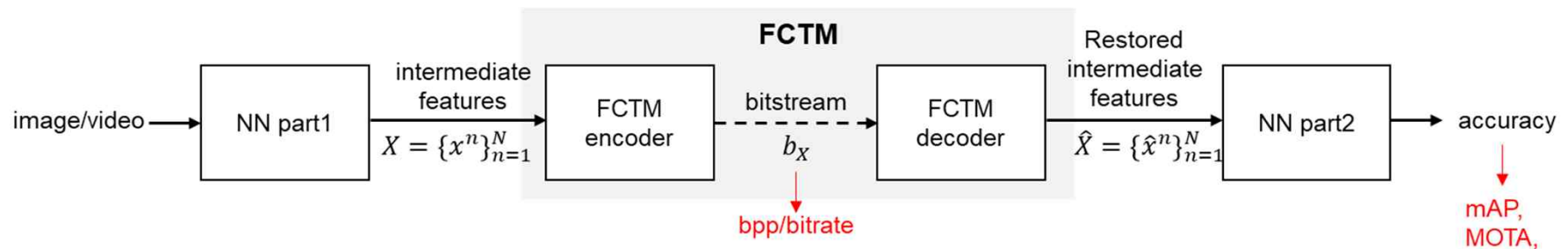
Contents

- Introduction to Feature Coding for Machines
- FCTM
 - Feature Channel Packing
 - Feature Channel Restoration
- Proposed Methods
 - Average Pooling-Based Channel Resize
 - Active Channel Residual-Based Channel Restoration
- Experimental Results
- Conclusion

1. Introduction to FCM

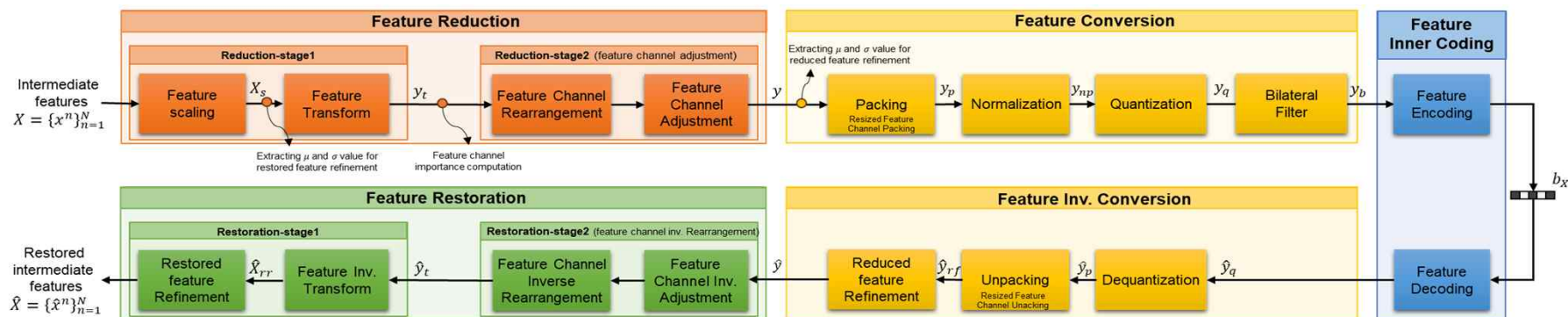
□ Feature Coding for Machines (FCM) in MPEG

- Machine vision applications are rapidly expanding across various AI-driven domains, demanding efficient feature transmission under bandwidth constraints
- As AI models grow larger (e.g., Vision Transformers, large-scale detection models), **transmitting raw intermediate features becomes increasingly costly**
 - Compression of intermediate features is critical for real-time inference in bandwidth-limited environments
- MPEG is developing a standard for image/video feature compression to support machine vision tasks called Feature Coding for Machines (FCM)
 - Feature Compression Test Model (FCTM) to evaluate standard candidate technologies
 - FCTM 1 (Oct. 2023, Hannover) → FCTM 10 (Jan. 2026, Online)



2. Overall Framework of FCTM

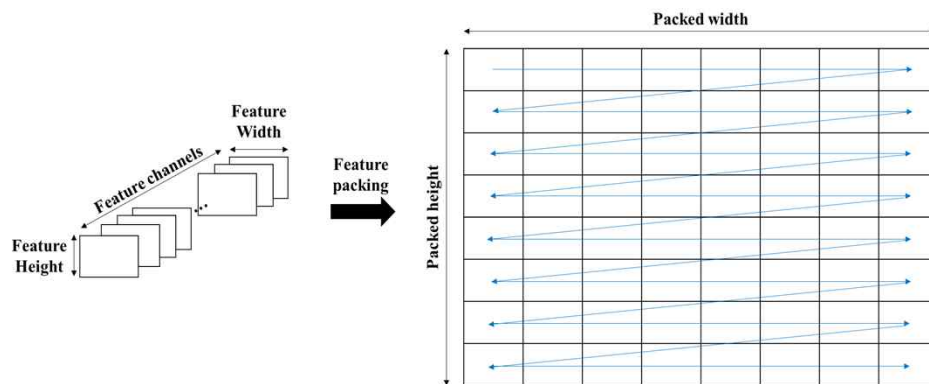
- ❑ Feature reduction
 - ❑ Multi-scale intermediate features are transformed into a single-scale representation via FENet with channel adjustment
- ❑ Feature conversion
 - ❑ Reduced features are packed into a 2D frame, followed by normalization and quantization for inner codec input
- ❑ Feature inner coding
 - ❑ Quantized feature frames are encoded/decoded using VVC codec
- ❑ Feature inverse conversion, feature restoration
 - ❑ Inverse operation of feature conversion and reduction



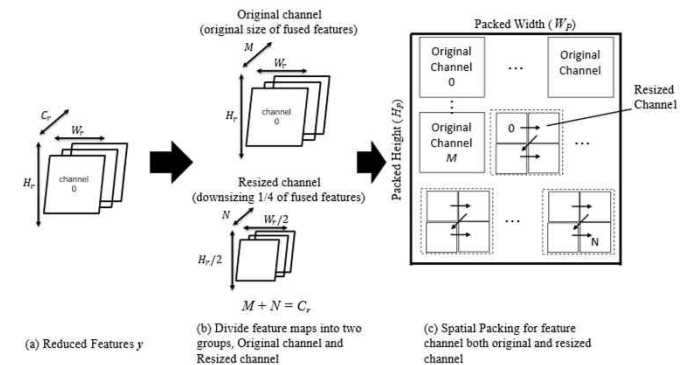
Overall framework of FCTM v10

Feature Channel Packing

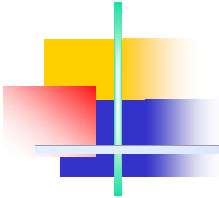
- Default feature packing
 - Converts reduced feature y (3D tensor $[C_r, H_t, W_t]$) into a single 2D frame $[1, H_p, W_p]$ channels are arranged in raster scan order on the 2D frame
 - Number of horizontal/vertical channels determined by $long_edge / short_edge$
- Resized feature channel packing
 - Downscales less important channels to 1/4 size before packing
 - 4 resized channels are merged into 1 channel: $C'_r = M + \lceil N/4 \rceil$
 - Down-sampling method: Bilinear interpolation



Spatial packing for the feature channel

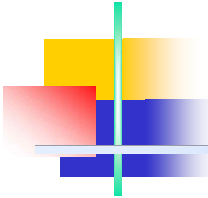


Pipeline of resized feature packing



Removed Channel Restoration

- ❑ Feature channel restoration (Decoder)
 - ❑ Channels restored to original positions using received rearrangement index
 - Output tensor initialized with mean value y_{mean} of restored channels
 - Active channels → filled with restored values
 - Removed channels → replaced with y_{mean}
 - ❑ Output: Restored Transformed Feature y_t , shape $[C_t, H_t, W_t]$
- ❑ Limitation of anchor method
 - ❑ Packing: resized channels down-sampled by bilinear interpolation
 - Simply reduces resolution **without preserving structural information**
 - ❑ Restoration: removed channels replaced with y_{mean} (mean of active channels)
 - **No spatial or inter-channel correlation** utilized



3. Proposed Method – Average Pooling-Based Channel Resize

- ❑ Existing method
 - ❑ Resized feature channels are down sampled using bilinear interpolation
 - ❑ Bilinear interpolation generates reduced features by sampling neighboring pixels with distance-based weights
 - ❑ For odd-sized feature maps, some spatial information is not reflected in the reduced feature representation due to non-uniform sampling locations
- ❑ Adaptive average pooling
 - ❑ Adaptive average pooling generates the target feature size through region-wise averaging
 - ❑ This approach provides stable feature resizing for odd-sized feature maps
 - ❑ Unlike bilinear interpolation, adaptive average pooling aggregates information from all input regions contributing to the target feature map

Average Pooling-Based Channel Resize

□ Feature size analysis

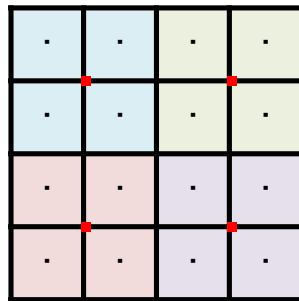
- Feature map sizes differ across datasets, and odd-sized dimensions are observed in SFU and PANDA
- Resizing differences between bilinear interpolation and adaptive average pooling appear only for odd-sized feature maps

	Dataset	Feature Size	Type
Object Detection	SFU	$13 \times 20/13 \times 21/25 \times 40$	Odd Included
Object Tracking	TVD	12×20	Even x Even
	HiEve	12×20	Even x Even
Semantic Segmentation	PANDA	12×21	Even x Odd

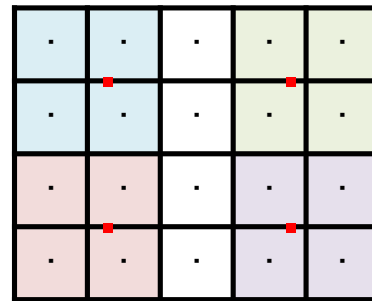
Size of single-scale feature channel

Average Pooling-Based Channel Resize

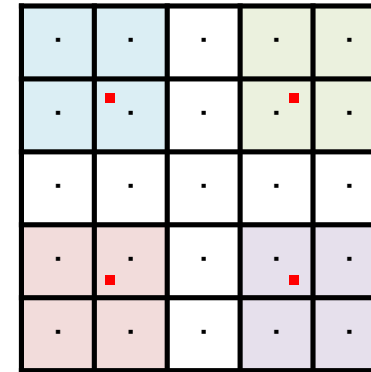
□ Bilinear interpolation



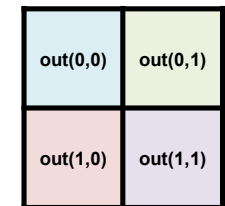
Even x Even



Even x Odd



Odd x Odd

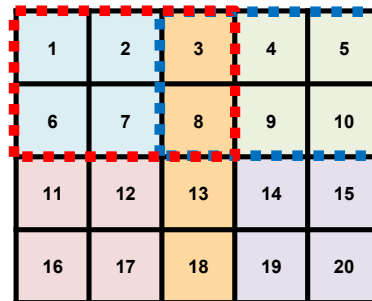


Output

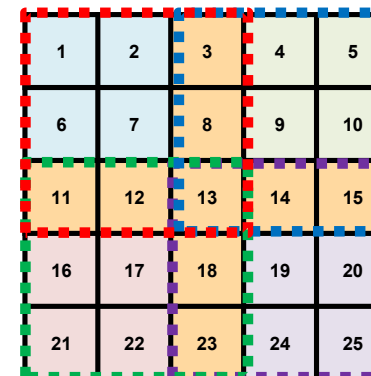
□ Adaptive average pooling



Even x Even



Even x Odd



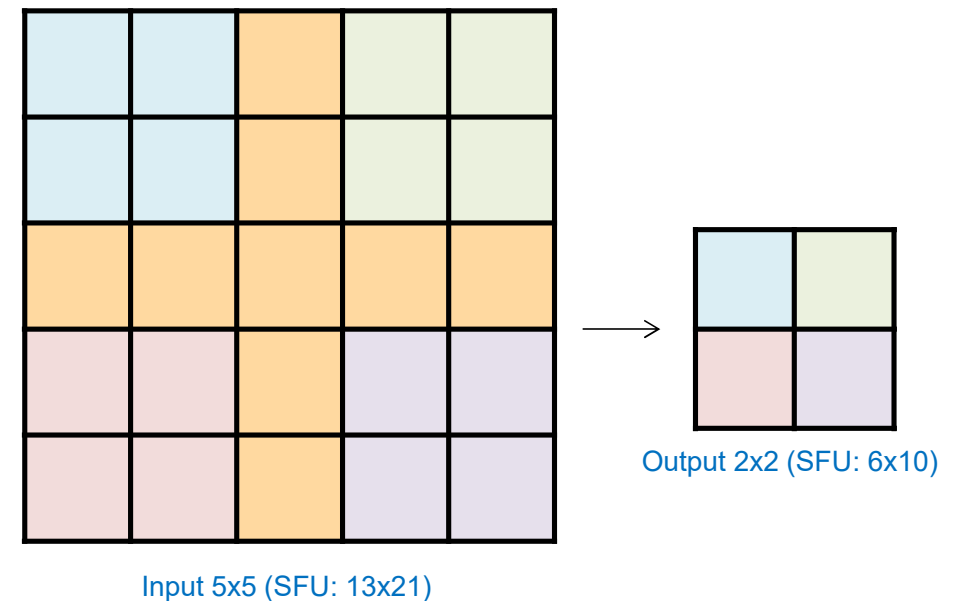
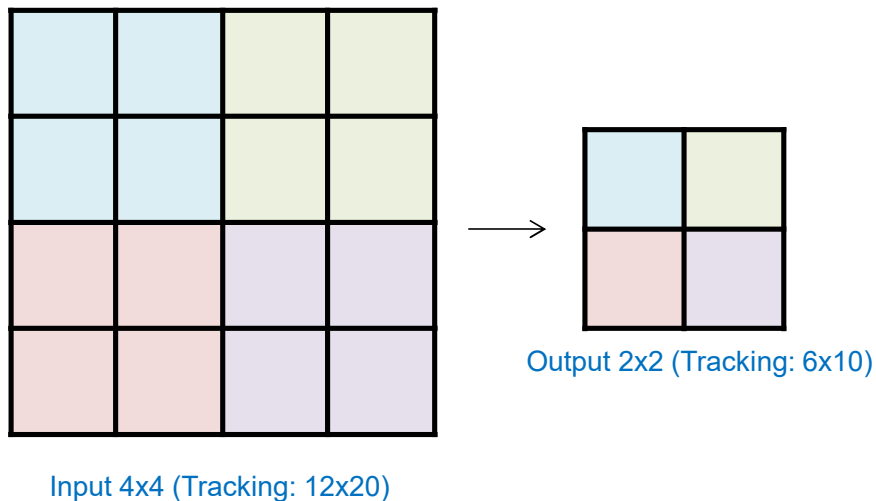
Odd x Odd

 : overlap

Average Pooling-Based Channel Resize

□ Adaptive average pooling

- Adaptive average pooling divides the input feature map into regions based on the target output size
- The average value of each region is then used to generate the resized feature map



4. Active Channel Residual-Based Channel Restoration

Anchor method

- Removed channels are reconstructed using a scalar mean value
- The same scalar mean is applied to all pixels in the removed channel
- This approach is simple but does not reflect local spatial variations

Active channel residual

- A residual term is defined using neighboring active channels
- The residual is computed as the pixel-wise difference between the right and left active channels
- The restoration value is generated by the equation

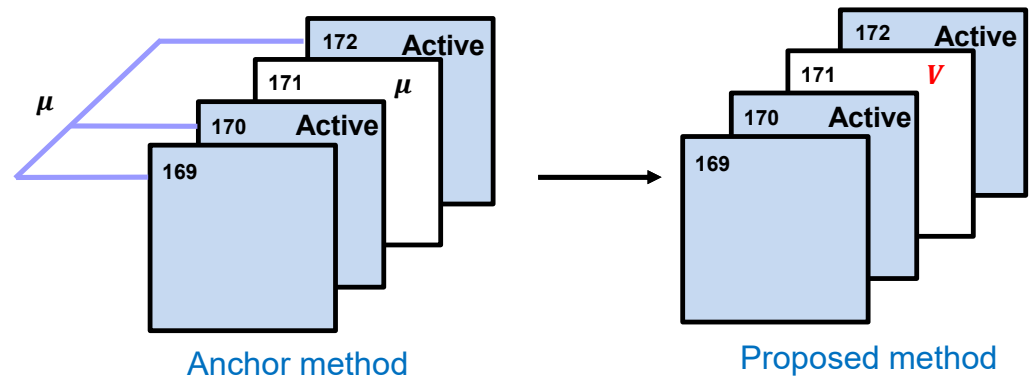
$$\text{Removed Channel} = \alpha \cdot (R - L)$$

$$\alpha = 0.05$$

R : Right Active Channel Pixel Value

L : Left Active Channel Pixel Value

α : Scaling Factor



5. Experimental Results

□ Test conditions

- The proposed methods were implemented in FCTM v10.0 and evaluated under the FCM CTTC

□ Tasks

- Object Detection: Locates objects with bounding boxes and classifies them
- Object Tracking: Follows specific objects through video frames, maintaining their identity
- Semantic Segmentation: Pinpoints objects' exact pixel boundaries

FCM Common Training and Test Conditions

Task	Dataset	Network	Split point	Rate measure	Task measure
Instance Segmentation	OpenImagesV6	MaskRCNN-X101-FPN	P-layer (P2-P5)	BPP	mAP @ 0.5
Object Detection	OpenImagesV6	FasterRCNN-X101-FPN	P-layer (P2-P5)	BPP	mAP @ 0.5
	SFU	FasterRCNN-X101-FPN	P-layer (P2-P5)	Kbps	mAP @ 0.5-0.95
Object Tracking	TVD	JDE-1088x608	Darknet-53	Kbps	MOTA
	HiEve	JDE-1088x608	Layers 75, 90, 105 ("ALT1")	Kbps	MOTA
Semantic Segmentation	Pandaset	Panoptic-FPN-R101	P-layer (P2-P5)	Kbps	mIoU

Inference Environment

	Inference environment
OS	Ubuntu 22.04.4 LTS
CPU	AMD Ryzen Threadripper 7960X
Python	3.8.19
Numpy	1.24.4
Torch	2.4.1+cpu
Torchvision	0.15.1+cpu
CompressAi	1.2.9.dev0
CompressAi-Vision	1.1.14.dev0
ffmpeg	4.2.2

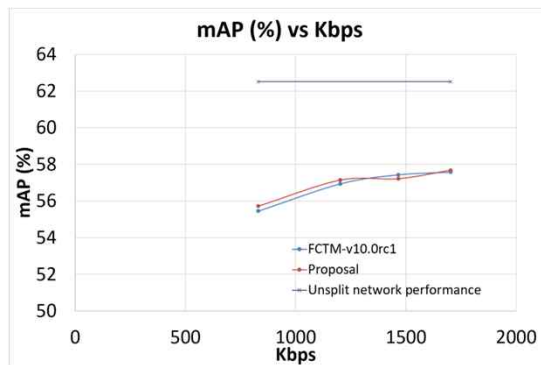
1) Resize Channel Pooling

- Experimental results – resize channel pooling
 - Comparison with the FCTM v10.0 anchor
 - The proposed method achieves an **overall BD-rate reduction of -1.42%**
 - No performance change was observed in object tracking tasks, where feature maps consist of even-sized dimensions (12×20)

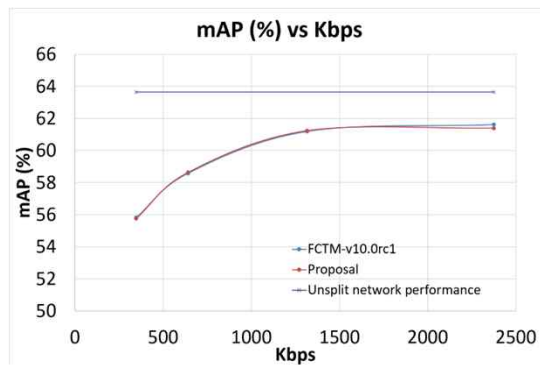
Proposal vs. FCTM-v10.0		
Task	Test Dataset	BD-rate [%]
Object Detection	SFU (Class A/B)	-6.37%
	SFU (Class C)	-1.76%
	SFU (Class D)	-4.06%
	SFU (Class A/B-no-rescale)	-3.98%
Object Tracking	TVD (OVERALL)	0.00%
	HiEve (1080p)	0.00%
	HiEve (720p)	0.00%
Semantic Segmentation	PANDAM1	1.48%
	PANDAM2	1.53%
	PANDAM3	-1.08%
	OVERALL	-1.42%

Resize Channel Pooling

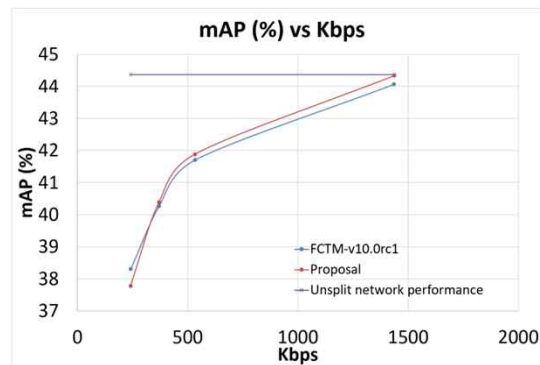
- RD-curves – resize channel pooling



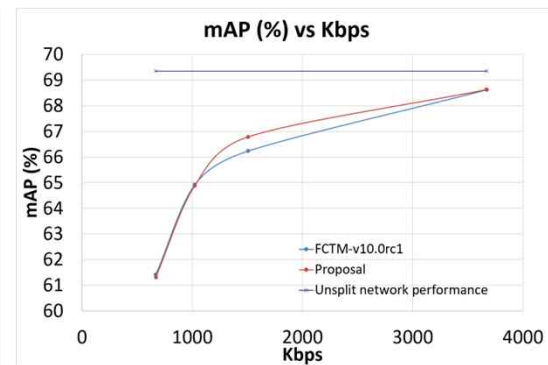
SFU Class A/B Detection



SFU Class C Detection



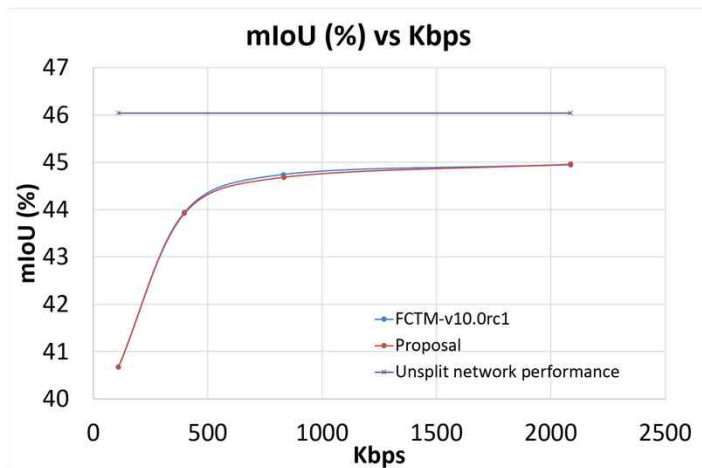
SFU Class D Detection



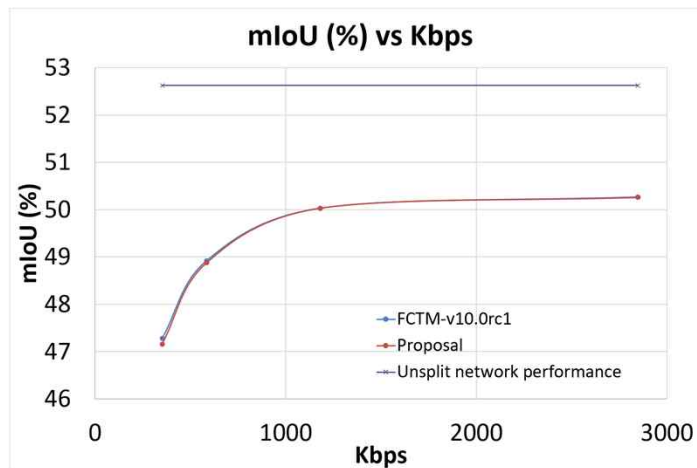
SFU Class A/B- no-rescale Detection

Resize Channel Pooling

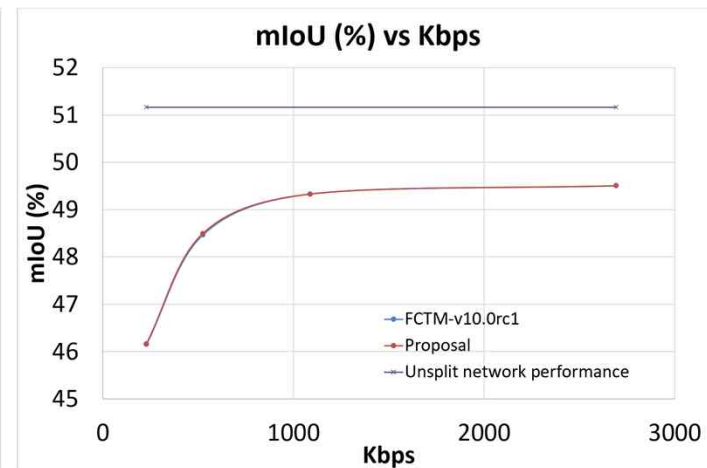
- RD-curves – resize channel pooling



PANDAM1 Segmentation



PANDAM2 Segmentation

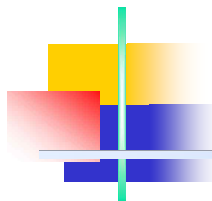


PANDAM3 Segmentation

2) Active Channel Residual

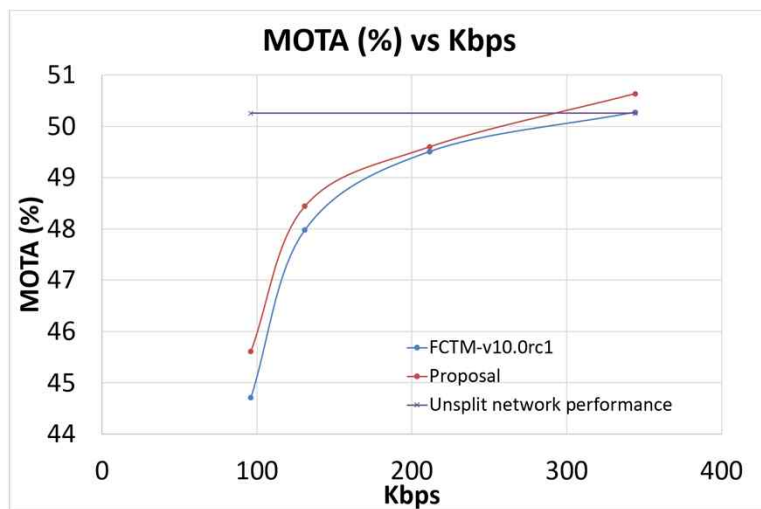
- Experimental Results – Active channel residual
 - Comparison with the FCTM v10.0 anchor
 - The proposed method achieves an **overall BD-rate reduction of -0.66%**
 - Active channel residual showed mixed results across datasets, indicating that neighboring channel differences can affect restoration performance

Proposal vs. FCTM-v10.0		
Task	Test Dataset	BD-rate [%]
Object Detection	SFU (Class A/B)	0.76%
	SFU (Class C)	-0.59%
	SFU (Class D)	-0.76%
	SFU (Class A/B-no-rescale)	-0.26%
Object Tracking	TVD (OVERALL)	-7.48%
	HiEve (1080p)	1.80%
	HiEve (720p)	-0.23%
Semantic Segmentation	PANDAM1	-0.04%
	PANDAM2	0.02%
	PANDAM3	0.16%
	OVERALL	-0.66%

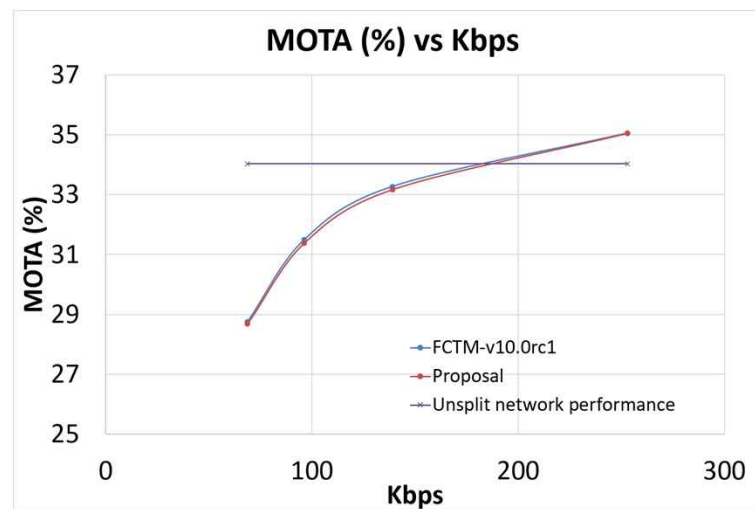


Active Channel Residual

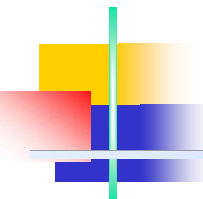
- RD-curves – active channel residual



TVD overall Tracking

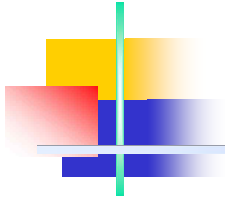


HiEve 1080p Tracking



6. Conclusions

- ❑ Proposed Methods for Enhancing FCTM Performance
 - ❑ Resize Channel Pooling
 - Replaced bilinear interpolation with adaptive average pooling in resize packing
 - Provides stable feature resizing for odd-sized feature maps
 - Utilizes the entire spatial region when generating resized features
 - ❑ Active Channel Residual
 - Replaced scalar mean with residual from neighboring active channels
 - Utilizes local differences between neighboring active channels
 - ❑ Conclusions
 - Both methods improve feature reconstruction without modifying the FCTM bitstream structure
 - Experimental results indicate the effectiveness of feature restoration techniques in FCTM
 - Resize channel processing and removed channel restoration can significantly affect the overall coding performance



Thank you for your attention!